TP – Stockage Temporaire des Données dans Android

BTS DAI – Développement Mobile

Objectifs du TP

À la fin de ce TP, l'étudiant sera capable de :
— Comprendre la différence entre stockage persistant et stockage temporaire
— Utiliser la mémoire volatile (RAM)
— Mettre en place un système de cache
— Gérer une session utilisateur dans une application Android
— Afficher des données temporaires dans une interface utilisant **StackView**

Introduction

Dans une application mobile, certaines données ne doivent pas forcément être sauvegardées de manière permanente dans une base de données.

On parle alors de **stockage temporaire**.

Ce stockage permet :
— d'améliorer les performances
— de réduire les accès réseau
— d'accélérer l'affichage des données

Dans ce TP nous allons étudier trois mécanismes :
— le **Cache**
— la **Mémoire volatile**
— la **Session utilisateur**

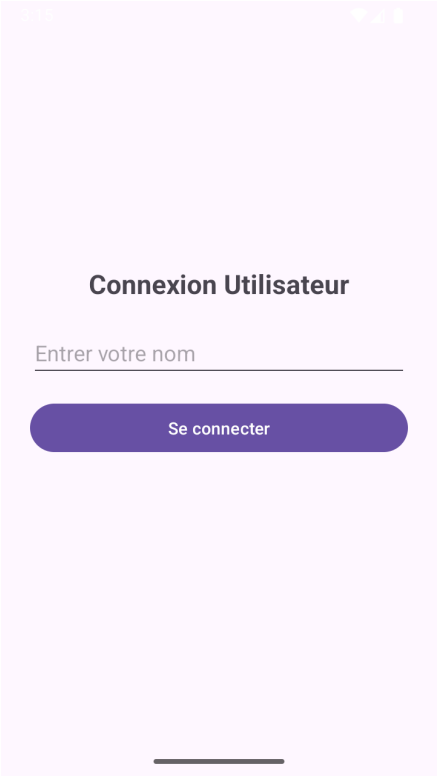
L'objectif de ce TP est de créer une mini application contenant :
— un écran de connexion
— un écran affichant des produits
— les produits sont stockés dans un cache
— l'utilisateur connecté est conservé dans une session
— les produits sont affichés dans un StackView

1 Partie 1 – Mémoire Volatile (RAM)

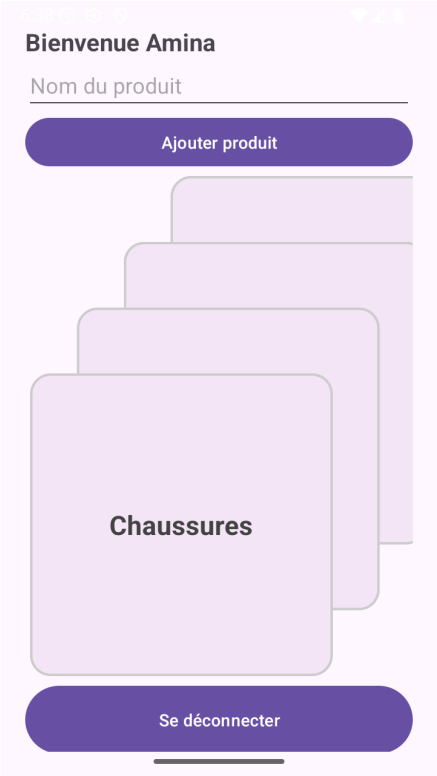
1.1 Principe

La mémoire volatile correspond aux variables stockées en RAM pendant l'exécution de l'application.

Caractéristiques :
— Très rapide
— Disponible uniquement pendant l'exécution
— Les données sont perdues lorsque l'application est fermée



(a) Liste StackView contenant les données



(b) Interface Login

FIGURE 1 – Interfaces de l'application

1.2 Exemple Théorique

Une variable Java peut être utilisée pour stocker temporairement une information.

2 Partie 4 – Session Utilisateur

2.1 Principe

Une session permet de conserver les informations d'un utilisateur pendant l'utilisation de l'application.

Exemples :

— utilisateur connecté
— panier d'achat
— préférences temporaires

La session est généralement stockée :

— en RAM
— dans une classe singleton

2.2 Travail demandé

Créer une application contenant :

— un champ EditText pour saisir le nom de l'utilisateur
— l'utilisateur saisit son nom, et clique sur un bouton "Se connecter"
— un TextView affichant l'utilisateur connecté

—
— le nom de l'utilisateur apparaît sur toutes les activités

Consignes

— créer une classe **SessionManager**
— stocker le nom de l'utilisateur dans une variable statique
— afficher l'utilisateur connecté dans l'activité principale

```
public class SessionManager {

    private  ______ String utilisateur;

    public static void setUtilisateur(String u){
        ______ = u;
    }

    public static String getUtilisateur(){
        return ______;
    }
    public static void logout(){
        ______;
    }

}
```

3 Partie 2 – Le Cache

3.1 Principe

Le cache permet de stocker temporairement des données afin d'éviter de les recalculer ou de les télécharger plusieurs fois.

Exemples :

— images téléchargées depuis internet
— informations produits
— résultats d'une API

Android propose une classe appelée :

LruCache

Elle supprime automatiquement les données les moins utilisées.

3.2 Travail demandé

Créer un système de cache pour stocker des produits.

Chaque produit possède :

— un identifiant
— un nom

Étapes

— créer un cache avec LruCache
— ajouter 5 produits dans le cache
— récupérer un produit par son identifiant

```java
public class CacheManager {

    public static ______<String,String> cache;

      public static ArrayList<String> produits = new ArrayList<>();

    public static void initCache(){
     if(cache == null){

        int cacheSize = 4 * 1024 * 1024;

        cache = new ______<>(cacheSize);

        cache.put("1","________");
        cache.put("2","________");
        cache.put("3","________");

    }}

}
```

4 Partie 3 : Connexion utilisateur

4.1 Partie théorique

Dans l'activité principale (MainActivity.java), nous récupérons le nom de l'utilisateur et nous l'enregistrons dans la session. Il faut commencer d'abord par initier le cache via la méthode initCache qu'on a crée dans la classe CacheManager.

```java
CacheManager.________;

btnLogin.setOnClickListener(v -> {

    String nom = editNom.getText().toString();

    SessionManager.setUtilisateur(nom);

    Intent i =________

    ________;

});
```

5 Partie 4 – Adapter personnalisé

Le StackView a besoin d'un **Adapter** pour afficher les données.

5.1 Travail demandé

Créer la classe **ProduitAdapter**, contenant une arraylist : ArrayList<String> produits ; et les méthode : getCount, getItem, getItemId et getView. Afficher les produits stockés dans le cache dans un StackView, via getView .

```
public ProduitAdapter(________, ________) {
        this.________ = ________;
        this.________ = ________;
        inflater = LayoutInflater.________;
    }
    @Override
    public int ________ {
        return produits.size();
    }

    @Override
    public Object getItem(int i) {
        return ________;
    }

    @Override
    public long ________ {
        return ________;
    }
    @Override
    public View getView(int i, View view, ViewGroup parent) {

        LayoutInflater ________ = ________;
        view = ________;

        TextView txtProduit = view.________(R.id.txtProduit);

        txtProduit.________(produits.________);

        return ________;
    }
```

6 Partie 5 – Affichage des données avec StackView

Pour afficher les produits, nous allons utiliser un composant Android appelé :
StackView
Il permet d'afficher les éléments comme une pile de cartes.

6.1 activity_produits.xml

```
<StackView
    android:id="@+id/stackView"
    android:layout_width="match_parent"
```

```
    android:layout_height="300dp"/>
```

Ajouter également un textView pour afficher un message de bienvenue, selon le nom de l'utilisateur. Un EditText pour ajouter de nouveaux produits, et un bouton pour activer l'insertion. A la fin, ajouter un bouton de déconnexion.

6.2 Initialisation des composants

Le StackView utilise un **Adapter** pour afficher les données. Créer la classe **ProduitsActivity** , et faîtes appel à toutes les vues nécessaires.

```
stackView = findViewById(R.id.stackView);
txtBienvenue = findViewById(R.id.txtBienvenue);
editProduit = findViewById(R.id.editProduit);
btnAjouter = findViewById(R.id.btnAjouter);
btnLogout = findViewById(R.id.btnLogout);
```

6.3 Affichage du message de bienvenue

6.3.1 Partie théorique

On récupère le nom de l'utilisateur depuis la session.

```
txtBienvenue.________;
```

6.4 Chargement des produits depuis le cache

Les produits sont stockés dans une liste globale :

```
produits = ________;
```

6.5 Ajout des produits dans la liste

6.5.1 Partie théorique

On récupère les produits depuis le cache.
Compléter :

```
produits.add(__________.cache.get("1"));
produits.add(__________.__________.get("2"));
produits.add(__________.cache.__________);
```

6.6 Association Adapter + StackView

L'adapter permet d'afficher les données dans le StackView. Compléter :

```
adapter = new __________(this, produits);
stackView.set__________(adapter);
```

6.7 Ajout d'un nouveau produit

On ajoute un produit dans la liste puis on met à jour l'affichage.
Compléter :

```
btnAjouter.setOnClickListener(v -> {
String nouveauProduit = __________.getText().toString();

if(!nouveauProduit.isEmpty()){

    produits.__________(nouveauProduit);

    adapter.________________(); //notifier l'adapter que les donné
       es ont changé.
    ________________ //ajouter une notification d'ajout
    editProduit.________________("""); //vider le champ
}});
```

6.8 Gestion de la déconnexion

6.8.1 Partie théorique

La déconnexion consiste à :
— supprimer la session utilisateur
— revenir à l'écran principal
— fermer l'activité actuelle
Compléter :

```
btnLogout.setOnClickListener(v -> {

    SessionManager.________();

    Intent intent = new Intent(
        ProduitsActivity.this,
        ____________.class
    );

    startActivity(intent);

    ______();

});
```

7 Questions de réflexion

1. Quelle est la différence entre mémoire volatile et stockage SQLite ?
2. Pourquoi utiliser un cache dans une application mobile ?
3. Que se passe-t-il si l'application est fermée lorsque les données sont uniquement en RAM ?
4. Dans quel cas utiliser une session utilisateur ?